R.M.K
GROUP OF ENGINEERING INSTITUTIONS

R.M.K
GROUP OF
INSTITUTIONS

R.M.K
GROUP OF INSTITUTIONS

R.M.K
GROUP OF
INSTITUTIONS

Please read this disclaimer before proceeding:

This document is confidential and intended solely for the educational purpose of RMK Group of Educational Institutions. If you have received this document through email in error, please notify the system manager. This document contains proprietary information and is intended only to the respective group / learning community as intended. If you are not the addressee you should not disseminate, distribute or copy through e-mail. Please notify the sender immediately by e-mail if you have received this document by mistake and delete this document from your system. If you are not the intended recipient you are notified that disclosing, copying, distributing or taking any action in reliance on the contents of this information is strictly prohibited.

22CS910 - DevOps

Department : Artificial Intelligence & Data Science

Batch / Year : 2022 - 2026 / IV

Created by : Dr. Gladiss Merlin N.R, ASP/ADS.

Date : 17.05.2025

R.M.K GROUP OF INSTITUTIONS

1. Contents

S. No.	Contents
1	Contents
2	Course Objectives
3	Prerequisites
4	Syllabus
5	Course Outcomes
6	CO-PO Mapping
7	Lecture Plan
8	Activity Based Learning
9	Lecture Notes
10	Assignments
11	Part- A Questions & Answers
12	Part-B Questions
13	Supportive Online Courses
14	Real Time Applications
15	Content beyond the Syllabus
16	Assessment Schedule
17	Prescribed Text books & Reference Books
18	Mini Project Suggestions

R.M.K GROUP OF INSTITUTIONS

Content – Unit I

S.No.	Contents
1	Software Development Methodologies - Operations Methodologies
2	Systems Methodologies
3	Development, Release, and Deployment Concepts
4	Infrastructure Concepts
5	What is DevOps? - DevOps importance and benefits
6	DevOps principles and practices - 7 C's of DevOps lifecycle for business agility
7	DevOps and continuous testing
8	How to choose right DevOps tools?
9	Challenges with DevOps implementation

2. Course Objectives

The Course will enable learners to:

- Bridge the gap between development and operations for faster, more reliable software releases.
- Automate software delivery with CI/CD pipelines.
- Package and deploy apps efficiently using Docker containers.
- Automate infrastructure with Infrastructure as Code (IaC).
- Monitor and troubleshoot applications in production.

R.M.K GROUP OF INSTITUTIONS

3. Prerequisites

22CS910 - DevOps

22IT910
Rest Application Development Using Spring Boot and JPA

22CS402
Web Development Frameworks

22CS301
Advanced Java Programming

22CS202
Java Programming

22CS101 Problem Solving Using C++
22CS102 Software Development Practices

R.M.K GROUP OF INSTITUTIONS

4. Syllabus

22CS910	DevOps	L	T	P	C
		3	0	0	3

OBJECTIVES:
The Course will enable learners to:

- ❖ Bridge the gap between development and operations for faster, more reliable software releases.
- ❖ Automate software delivery with CI/CD pipelines.
- ❖ Package and deploy apps efficiently using Docker containers.
- ❖ Automate infrastructure with Infrastructure as Code (IaC).
- ❖ Monitor and troubleshoot applications in production.

UNIT I	Introduction to DEVOPS	9

Software Development Methodologies - Operations Methodologies - Systems Methodologies - Development, Release, and Deployment Concepts - Infrastructure Concepts. What is DevOps? - DevOps importance and benefits -DevOps principles and practices - 7 C's of DevOps lifecycle for business agility - DevOps and continuous testing. How to choose right DevOps tools? - Challenges with DevOps implementation.

UNIT II	Version Control with GIT	9

Build production-grade applications – MYSQL - mapping Java classes to relational database - Introduction to Git version control system - Git commands for basic operations (clone, commit, push, pull) - Branching and merging strategies - Collaboration using Git workflows.

UNIT III	Continuous Integration and Delivery (CI/CD)	9

Introduction to CI/CD pipelines - Benefits of CI/CD for faster deployments - Setting up a CI/CD pipeline with Jenkins - Automating builds, tests, and deployments.

UNIT IV	Containerization with Docker	9

Introduction to containerization and its benefits - Understanding Docker concepts: images, containers, registries - Building and managing Docker containers - Docker Compose for multi- container applications - Introduction to container orchestration with Docker Swarm or Kubernetes.

UNIT V	Infrastructure as Code (IaC) and Monitoring	9

Introduction to Infrastructure as Code (IaC) - Benefits of using IaC for repeatable infrastructure provisioning - Learning IaC with Terraform - Setting up infrastructure configurations with Terraform - Introduction to monitoring and logging tools for applications - Alerting and troubleshooting techniques.

TOTAL: 45 PERIODS

R.M.K GROUP OF INSTITUTIONS

4. Syllabus Contd...

OUTCOMES:

Upon completion of the course, the students will be able to:

CO1: Understand the core principles and philosophies of DevOps.
CO2: Implement version control systems for code management and collaboration.
CO3: Automate software delivery pipelines using CI/CD tools.
CO4: Utilize containerization technologies for packaging and deploying applications.
CO5: Configure infrastructure as code (IaC) for repeatable deployments.
CO6: Monitor and maintain applications in a production environment.

TEXTBOOKS:

1. Deepak Gaikwad, Viral Thakkar, "DevOps Tools: from Practitioner's Point of View", Wiley, 2019.
2. Jennifer Davis, Ryn Daniels, "Effective DevOps", O'Reilly Media, 2016.

REFERENCES:

1. Gene Kim, Jez Humble, Patrick Debois, "The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations", IT Revolution Press, 2016.
2. Jez Humble, Gene Kim, "Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation", Addison-Wesley, 2010.
3. Yevgeniy Brikman, "Terraform: Up & Running: Writing Infrastructure as Code", O'Reilly Media, 2019.
4. Joseph Muli, "Beginning DevOps with Docker", Packt Publishing, 2018.

R.M.K GROUP OF INSTITUTIONS

5. Course Outcomes

Upon completion of the course, the students will be able to:

CO1: Understand the core principles and philosophies of DevOps.

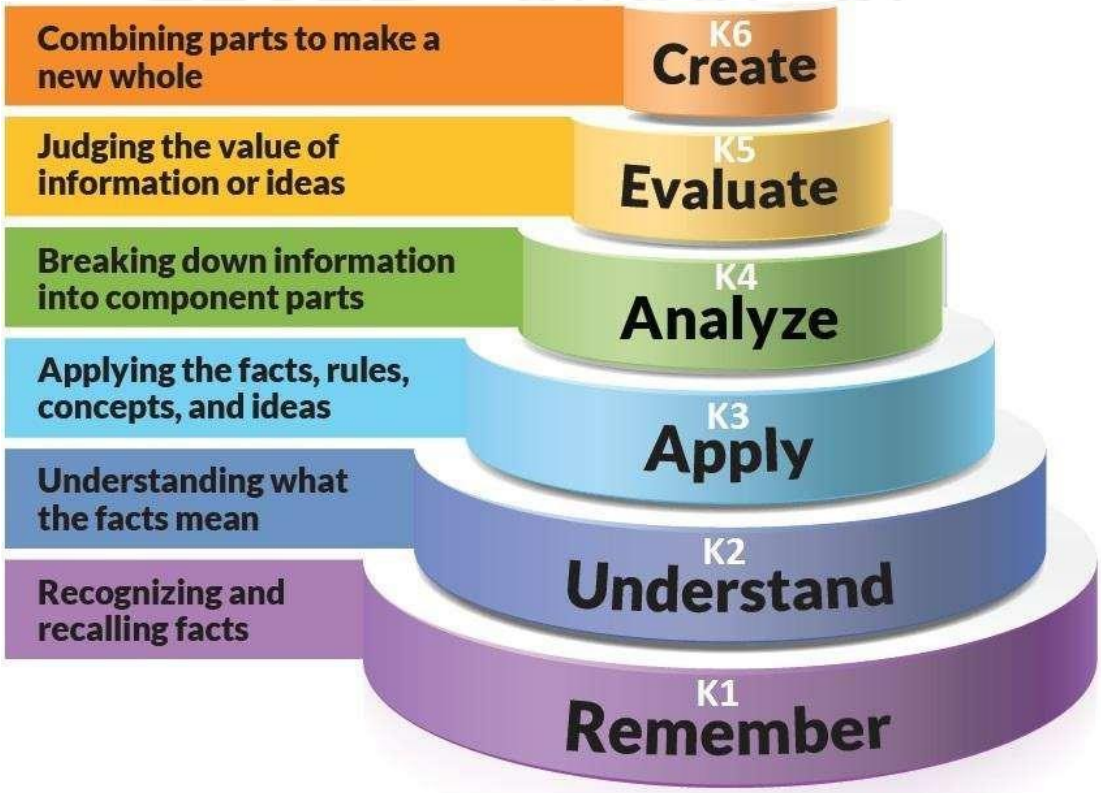
CO2: Implement version control systems for code management and collaboration.

CO3: Automate software delivery pipelines using CI/CD tools.

CO4: Utilize containerization technologies for packaging and deploying applications.

CO5: Configure infrastructure as code (IaC) for repeatable deployments.

CO6: Monitor and maintain applications in a production environment.



R.M.K GROUP OF INSTITUTIONS

6. CO - PO Mapping

	POs and PSOs														
COs	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO1	PSO2	PSO3
CO1	3	3	3		2			2	2	1		1			
CO2	3	3	3		2			2	2	1		1			
CO3	3	3	3		2			2	2	1		1			
CO4	3	3	3		2			2	2	1		1			
CO5	3	3	3		2			2	2	1		1			

R.M.K GROUP OF INSTITUTIONS

7. Lecture Plan - Unit I

S. No.	Topic	No. of Periods	Proposed Date	Actual Lecture Date	Pertaining CO	Taxonomy Level	Mode of Delivery
1	Software Development Methodologies - Operations Methodologies	1			CO1	K2	Chalk & Talk
2	Systems Methodologies	1			CO1	K3	Chalk & Talk
3	Development, Release, and Deployment Concepts	1			CO1	K3	Chalk & Talk
4	Infrastructure Concepts	1			CO1	K2	Chalk & Talk
5	What is DevOps? - DevOps importance and benefits	1			CO1	K3	Chalk & Talk
6	DevOps principles and practices - 7 C's of DevOps lifecycle for business agility	1			CO1	K3	Chalk & Talk
7	DevOps and continuous testing	1			CO1	K2	Chalk & Talk
8	How to choose right DevOps tools?	1			CO1	K3	Chalk & Talk
9	Challenges with DevOps implementation	1			CO1	K2	Chalk & Talk

8. Activity Based Learning

Learning Method	Activity
Learn by Solving Problems	Tutorial Sessions available in RMK Nextgen
Learn by Questioning	Quiz / MCQ Using RMK Nextgen App
Learn by doing Hands-on	Practice available in RMK Nextgen

RMK Nextgen

R.M.K GROUP OF INSTITUTIONS

9. Lecture Notes

Introduction to DEVOPS

Software Development Methodologies - Operations Methodologies - Systems Methodologies - Development, Release, and Deployment Concepts - Infrastructure Concepts. What is DevOps? - DevOps importance and benefits -DevOps principles and practices - 7C's of DevOps lifecycle for business agility - DevOps and continuous testing. How to choose right DevOps tools? - Challenges with DevOps implementation.

R.M.K GROUP OF INSTITUTIONS

1.1 Software Development Methodologies

Software development methodology is a set of practices, processes, and principles that guide the development of software. It covers the entire software development process, from initial planning to maintenance.

Some Traditional software development methodologies include:

Waterfall methodology : A traditional, linear approach that completes each phase of the process before moving on to the next. The classical waterfall model divides the life cycle into a set of phases. This model considers that one phase can be started after the completion of the previous phase. That is the output of one phase will be the input to the next phase. Thus the development process can be considered as a sequential flow in the waterfall.

Spiral model : A flexible model that emphasizes repetition through planning, design, build, and test phases.

Incremental model : Allows for early delivery of working software, and provides opportunities for user feedback at each iteration.

Iterative model : Emphasizes repetition by creating a version quickly and for relatively little cost, then testing and improving it through rapid and successive versions

V-model : A disciplined model that is easy to understand and makes project management easier. It is a good choice for software where downtimes and failures are unacceptable

Kanban : Easy to implement and understand, and allows for quick reprioritization in order to adapt changes according to market demand.

Dynamic systems model : An iterative and incremental approach that emphasizes continuous user involvement.

Prototype model : Used when the customers do not know the exact project requirements beforehand

R.M.K GROUP OF INSTITUTIONS

Some Agile software development methodologies include:

Rapid Application Development (RAD) : Focuses on quickly building prototypes based on user feedback. It emphasizes active user involvement and incremental development.

Scrum : A popular agile framework that uses an iterative approach to manage and deliver complex projects. It emphasizes collaboration, transparency, and adaptability.

DevOps : Combines development and operations teams to improve efficiency and promote collaboration. It works well for automating testing and delivery of complete processes.

Feature Driven Development (FDD) : An Agile-based method that aims to avoid confusion that can lead to expensive rework.

Extreme Programming (XP) : Also known as Agile methodology, its main goal is to create a fully-functional product while reducing the cost of non-essentials.

Prototyping : Involves building prototypes quickly based on a model. It allows for demand modifications and showing features during the project life cycle.

Lean development : Focuses on delivering maximum customer value while reducing waste and inefficiencies.

R.M.K GROUP OF INSTITUTIONS

Water Fall Methodology

The waterfall model is a software development model used in the context of large, complex projects, typically in the field of information technology. It is characterized by a structured, sequential approach to project management and software development.

The waterfall model is useful in situations where the project requirements are well-defined and the project goals are clear.

Features of Waterfall Model

Following are the features of the waterfall model:

Sequential Approach: The waterfall model involves a sequential approach to software development, where each phase of the project is completed before moving on to the next one.

Document-Driven: The waterfall model depended on documentation to ensure that the project is well-defined and the project team is working towards a clear set of goals.

Quality Control: The waterfall model places a high emphasis on quality control and testing at each phase of the project, to ensure that the final product meets the requirements and expectations of the stakeholders.

Rigorous Planning: The waterfall model involves a careful planning process, where the project scope, timelines, and deliverables are carefully defined and monitored throughout the project lifecycle.

Importance of Waterfall Model

Following are the importance of waterfall model:

Clarity and Simplicity: The linear form of the Waterfall Model offers a simple and unambiguous foundation for project development.

Clearly Defined Phases: The Waterfall Model phases each have unique inputs and outputs, guaranteeing a planned development with obvious checkpoints.

R.M.K GROUP OF INSTITUTIONS

Documentation: A focus on thorough documentation helps with software comprehension, maintenance, and future growth.

Stability in Requirements: Suitable for projects when the requirements are clear and stable, reducing modifications as the project progresses.

Resource Optimization: It encourages effective task-focused work without continuously changing contexts by allocating resources according to project phases.

Relevance for Small Projects: Economical for modest projects with simple specifications and minimal complexity.

Phases of Waterfall Model

The Waterfall Model has six phases which are:

Requirements: The first phase involves gathering requirements from stakeholders and analyzing them to understand the scope and objectives of the project.

Design: Once the requirements are understood, the design phase begins. This involves creating a detailed design document that outlines the software architecture, user interface, and system components.

Development: The Development phase include implementation involves coding the software based on the design specifications. This phase also includes unit testing to ensure that each component of the software is working as expected.

Testing: In the testing phase, the software is tested as a whole to ensure that it meets the requirements and is free from defects.

Deployment: Once the software has been tested and approved, it is deployed to the production environment.

Maintenance: The final phase of the Waterfall Model is maintenance, which involves fixing any issues that arise after the software has been deployed and ensuring that it continues to meet the requirements over time.

R.M.K GROUP OF INSTITUTIONS

Agile Software Development Methodology

Agile Software Development is a software development methodology that values flexibility, collaboration, and customer satisfaction. It is based on the Agile Manifesto, a set of principles for software development that prioritize individuals and interactions, working software, customer collaboration, and responding to change.

Agile Software Development is an iterative and incremental approach to software development that emphasizes the importance of delivering a working product quickly and frequently. It involves close collaboration between the development team and the customer to ensure that the product meets their needs and expectations.

Why Agile is Used?

Creating Tangible Value: Agile places a high priority on creating tangible value as soon as possible in a project. Customers can benefit from early delivery of promised advantages and opportunity for prompt feedback and modifications.

Concentrate on Value-Added Work: Agile methodology promotes teams to concentrate on producing functional and value-added product increments, hence reducing the amount of time and energy allocated to non-essential tasks.

Agile as a Mindset: Agile represents a shift in culture that values adaptability, collaboration, and client happiness. It gives team members more authority and promotes a cooperative and upbeat work atmosphere.

Quick Response to Change: Agile fosters a culture that allows teams to respond swiftly to constantly shifting priorities and requirements. This adaptability is particularly useful in sectors of the economy or technology that experience fast changes.

R.M.K GROUP OF INSTITUTIONS

Regular Demonstrations: Agile techniques place a strong emphasis on regular demonstrations of project progress. Stakeholders may clearly see the project's status, upcoming problems, and upcoming new features due to this transparency.

Cross-Functional Teams: Agile fosters self-organizing, cross-functional teams that share information effectively, communicate more effectively and feel more like a unit.

Twelve Principles of Agile Software Development

1. Ensuring customer satisfaction through the early delivery of software.
2. Being open to changing requirements in the stages of the development.
3. Frequently delivering working software with a main focus on preference for timeframes.
4. Promoting collaboration between business stakeholders and developers as an element.
5. Structuring the projects around individuals. Providing them with the necessary environment and support.
6. Prioritizing face to face communication whenever needed.
7. Considering working software as the measure of the progress.
8. Fostering development by allowing teams to maintain a pace indefinitely.
9. Placing attention on excellence and good design practices.
10. Recognizing the simplicity as crucial factor aiming to maximize productivity by minimizing the work.
11. Encouraging self organizing teams as the approach to design and build systems.
12. Regularly reflecting on how to enhance effectiveness and to make adjustments accordingly.

R.M.K GROUP OF INSTITUTIONS

The Agile Software Development Process

Requirements Gathering: The customer's requirements for the software are gathered and prioritized.

Planning: The development team creates a plan for delivering the software, including the features that will be delivered in each iteration.

Development: The development team works to build the software, using frequent and rapid iterations.

Testing: The software is thoroughly tested to ensure that it meets the customer's requirements and is of high quality.

Deployment: The software is deployed and put into use.

Maintenance: The software is maintained to ensure that it continues to meet the customer's needs and expectations.

Agile Software Development is widely used by software development teams and is considered to be a flexible and adaptable approach to software development that is well-suited to changing requirements and the fast pace of software development.

Agile is a time-bound, iterative approach to software delivery that builds software incrementally from the start of the project, instead of trying to deliver all at once.

Advantages Agile Software Development

Increased collaboration and communication: Agile Software Development Methodology emphasize collaboration and communication among team members, stakeholders, and customers. This leads to improved understanding, better alignment, and increased buy-in from everyone involved.

Flexibility and adaptability: Agile methodologies are designed to be flexible and adaptable, making it easier to respond to changes in requirements, priorities, or market conditions. This allows teams to quickly adjust their approach and stay focused on delivering value.

R.M.K GROUP OF INSTITUTIONS

Improved quality and reliability: Agile methodologies place a strong emphasis on testing, quality assurance, and continuous improvement. This helps to ensure that software is delivered with high quality and reliability, reducing the risk of defects or issues that can impact the user experience.

Enhanced customer satisfaction: Agile methodologies prioritize customer satisfaction and focus on delivering value to the customer. By involving customers throughout the development process, teams can ensure that the software meets their needs and expectations.

Increased team morale and motivation: Agile methodologies promote a collaborative, supportive, and positive work environment. This can lead to increased team morale, motivation, and engagement, which can in turn lead to better productivity, higher quality work, and improved outcomes.

Disadvantages Agile Software Development

Lack of predictability: Agile Development relies heavily on customer feedback and continuous iteration, which can make it difficult to predict project outcomes, timelines, and budgets.

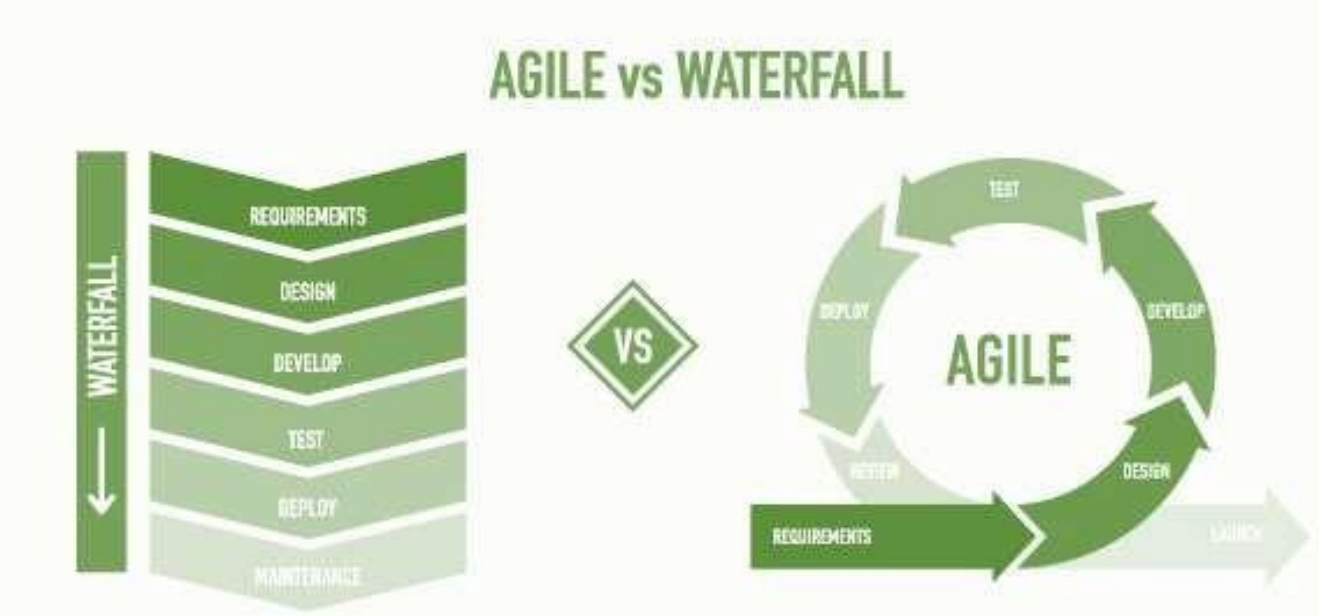
Limited scope control: Agile Development is designed to be flexible and adaptable, which means that scope changes can be easily accommodated. However, this can also lead to scope creep and a lack of control over the project scope.

Lack of emphasis on testing: Agile Development places a greater emphasis on delivering working code quickly, which can lead to a lack of focus on testing and quality assurance. This can result in bugs and other issues that may go undetected until later stages of the project.

Risk of team burnout: Agile Development can be intense and fast-paced, with frequent sprints and deadlines. This can put a lot of pressure on team members and lead to burnout, especially if the team is not given adequate time for rest and recovery.

R.M.K GROUP OF INSTITUTIONS

Lack of structure and governance: Agile Development is often less formal and structured than other development methodologies, which can lead to a lack of governance and oversight. This can result in inconsistent processes and practices, which can impact project quality and outcomes.



GROUP OF INSTITUTIONS

Agile

This approach focuses on continuous releases and customer feedback. It's characterized by adaptability, transparency, collaboration, and efficient communication. Agile allows for changes along the way, and it's beneficial for software projects where the structure isn't particularly defined.

Waterfall

This approach is linear and sequential, and it requires defining and documenting all requirements, design, and specifications before starting coding. It's best for physical structure projects where the process for doing it right is important.

Scrum

Scrum is a management framework that teams use to self-organize and work towards a common goal. It describes a set of meetings, tools, and roles for efficient project delivery. Much like a sports team practicing for a big match, Scrum practices allow teams to self-manage, learn from experience, and adapt to change. Software teams use Scrum to solve complex problems cost effectively and sustainably. Some characteristics of Scrum include: Iterative and incremental approach, Short sprints, Constant feedback, Fast moving and money efficient, and Customer satisfaction is very important. A project management framework that helps teams work together to achieve a common goal:

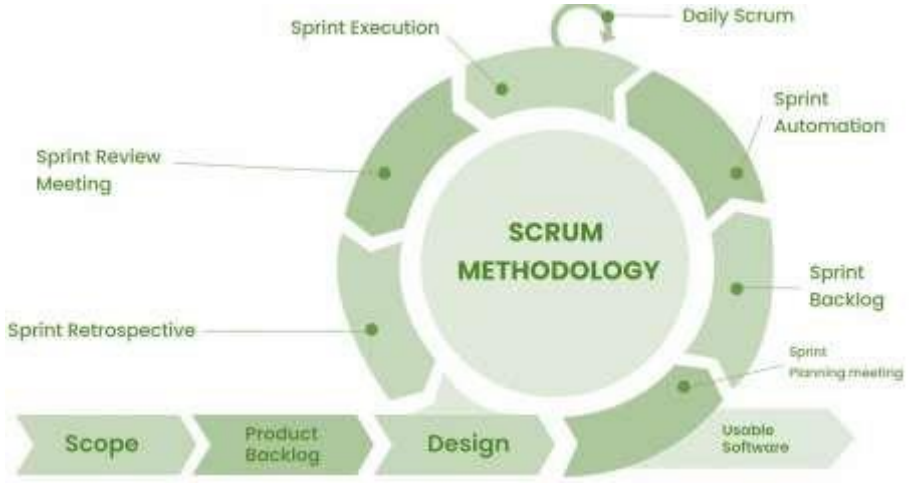
Break work into sprints : Teams divide work into goals to be completed within time-boxed iterations called sprints. Sprints are typically two weeks long, but can be up to one month.

Self-organize : Teams learn from experience, adapt to change, and reprioritize tasks as needed.

Reflect on progress : Teams regularly review their progress and use the insights to plan for the future.

Communicate: Teams have frequent face-to-face conversations to prevent miscommunication.

Surface issues : Teams work in an environment where everyone is aware of issues that might be affecting the team's success.



Some benefits of Scrum include:

Accelerates software delivery

Fosters teamwork, communication, and transparency

Reduced documentation overhead

Operations Methodology

Operations management (OM) is the practice of handling day-to-day business functions in a way that is efficient and maximizes profitability. OM teams try to balance costs with revenue to achieve the highest possible net operating profit.

IT operations management: Performs daily activities needed to manage IT services and support infrastructure (e.g., facilities management) to ensure the overall success of operations.

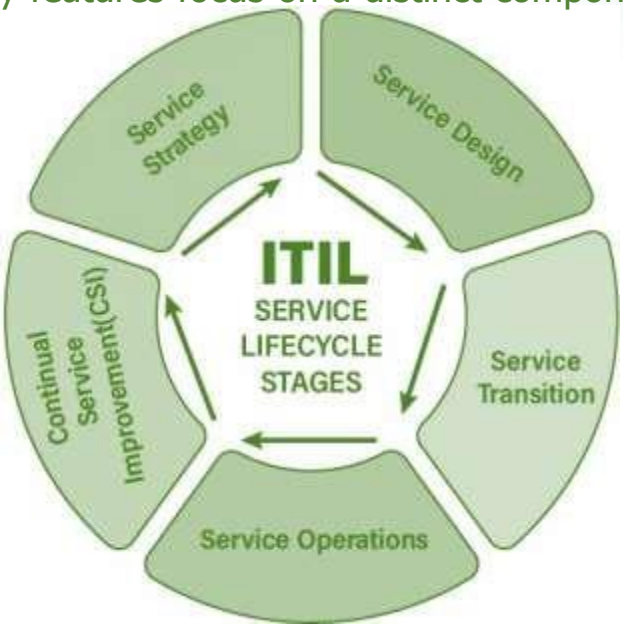
ITIL stands for Information Technology Infrastructure Library. ITIL was developed in the 1980s by the United Kingdom's Central Computer and Telecommunications Organization (CCTA), a government agency responsible for providing IT assistance to other ministries.

ITIL methodology is a well-known IT best practice that helps firms align their IT services with ITIL methodology consumer and business goals. IT-related assets, availability, and resources that provide value and advantages to clients are examples of services.

R.M.K GROUP OF INSTITUTIONS

Basic Features of ITIL Methodology

The ITIL agile methodology features focus on a distinct component of the IT service's lifecycle:



1. Service Strategy

The goal of the Service strategy is to establish an ITIL service lifecycle strategy. The strategy should be in line with the company's goals. This component's utility and warranty ensure that the service is up to standard and suitable for use, respectively. It is critical to do this since these two components give value to the supply of services to clients.

2. Service Design

The Service Design lifecycle stage is concerned with the creation of services and all enabling components for their implementation in the live environment. The "Four Ps of Service Design" are areas that should be considered while creating a service. They are as follows:

People

Processes

Products

Partners

3. Service Transition

The Service Transition process's goal is to design and deploy IT services while ensuring that modifications to services and Service Management procedures are conducted cohesively.

R.M.K GROUP OF INSTITUTIONS

4. Service Operations

Service Operations is concerned with achieving end-user expectations while balancing expenses and identifying possible difficulties. Fulfilling customer requests, addressing service outages, troubleshooting, and performing normal operational duties are all part of the Service Operations process. This is the only category of the five that includes functions and processes.

5. Continual Service Improvement (CSI)

The goal of this phase is to apply quality management systems to learn from previous achievements and mistakes. It strives to continuously enhance the efficacy and efficiency of IT processes and services following the ISO 2000 concept of continuous improvement. In this domain, there is just one method with seven steps:

1. Identifying solutions for improvement
2. Specifying the metrics to be used
3. Data collection
4. Data analysis
5. presenting and utilizing data-derived information
6. Processing of data
7. Making use of the data to improve services

Benefits of ITIL Framework

The benefits of the ITIL framework extend beyond improving service delivery. Implementing ITIL's best practices can help organizations:

- Optimize IT operations and reduce costs
- Align IT with business objectives and goals
- Improve customer satisfaction and enhance the user experience
- Maximize the value of IT investments
- Create a culture of continuous improvement

R.M.K GROUP OF INSTITUTIONS

Real Time Examples

Example 1: A multinational corporation can use the ITIL framework to standardize its IT operations across different regions, resulting in a potential 20% reduction in incident resolution times.

Example 2: A healthcare provider can implement ITIL frameworks to improve its service delivery, resulting in enhanced patient care and a significant reduction in system downtimes.

COBIT - Control Objectives for Information and Related Technology

COBIT is a collection of best practices and principles, along with tools and models for making sure IT resources and processes align with and support business goals. It was first introduced by the Information Systems Audit and Control Association (ISACA) in 1996, and has gone through many rounds of development since

COBIT (Control Objectives for Information and Related Technologies) is a framework that helps organizations manage and govern their IT resources. It provides a set of guidelines, principles, and best practices to help organizations:

1. Define and implement IT goals, policies, processes, and controls
2. Ensure IT complies with relevant standards and regulations
3. Assess and benchmark IT capabilities

Identify gaps and improvement opportunities

What are the benefits of COBIT?

COBIT offers models to help maximize the value and trust in IT, and these extended guidelines provide security, risk, reward, business and IT consulting professionals with a more extended framework to help in delivering and maintaining enterprise objectives and strategies.

Benefits of COBIT are listed below:

- Helps achieve operational excellence through efficient and effective application of technology and trustworthiness.
- Optimizes the cost of IT services and technology.
- Aids in managing and maintaining IT-related risk.
- Ensures the use of IT effectively and innovatively to align with strategic business goals.
- Maintains high-quality information to help support business decisions.
- Offers full support for IT firms that comply with business-oriented policies, regulations, relevant laws, and contractual agreements.

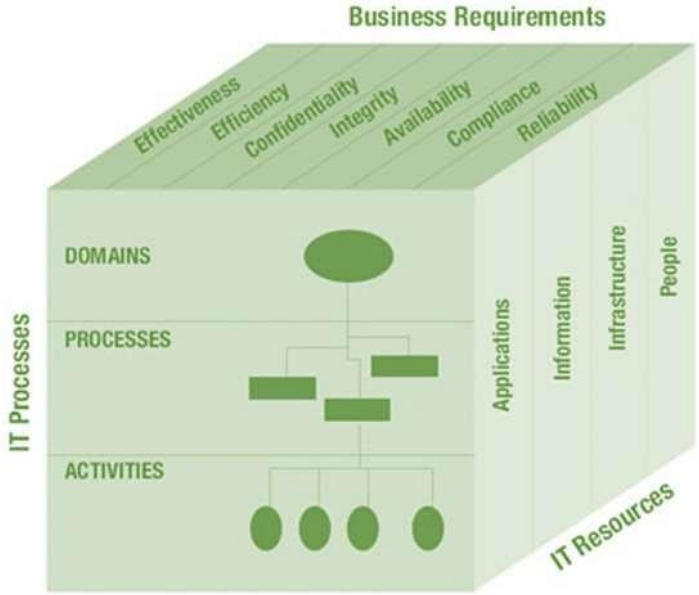
COBIT structure

From the highest level, COBIT creates a three-level structure comprised of the following segments:

1. Business requirements (information criteria of COBIT), including metrics such as integrity, effectiveness, availability, efficiency, compliance, confidentiality, and reliability
2. IT resources, including infrastructure, applications, information, and people and
3. IT processes, divided into domains processes

The COBIT cube

The affiliations between these components are exemplified by the COBIT cube.



Principles of COBIT

The six COBIT principles lay out the important aspects of IT enterprise governance:

- **Meet the needs of stakeholders** by aligning IT governance systems with their requirements, creating value through balancing benefits, risks, and resources.
- **Effective IT governance systems should be holistic**, composed of several separate components so that all factors are considered.
- Ensure that your governance system is dynamic by continuing to assess and improve it to stay relevant as the business and environment evolves.
- **IT management that focuses on operations and IT governance that focuses on oversight should be separate**, with defined roles that do not overlap.
- **Define and prioritize IT governance system components, tailored to meet the needs of the enterprise.**
- Cover all IT functions, data, and technology as a unified whole within the governance system, to meet the goals set by the enterprise.

Components of COBIT

COBIT components include:

Framework : Organize and categorize IT governance objectives and good practices by IT domains and processes before associating them with their respective business requirements.

Process descriptions : A reference process model and common language for everyone in an enterprise.

Control objectives : Use this complete set of high-level requirements for effective control of each IT process.

Management guidelines : Assign responsibility, agree on objectives, measure performance, and illustrate interrelationship with other processes.

Maturity models : Assess maturity and capability per process and helps to address gaps.

R.M.K GROUP OF INSTITUTIONS

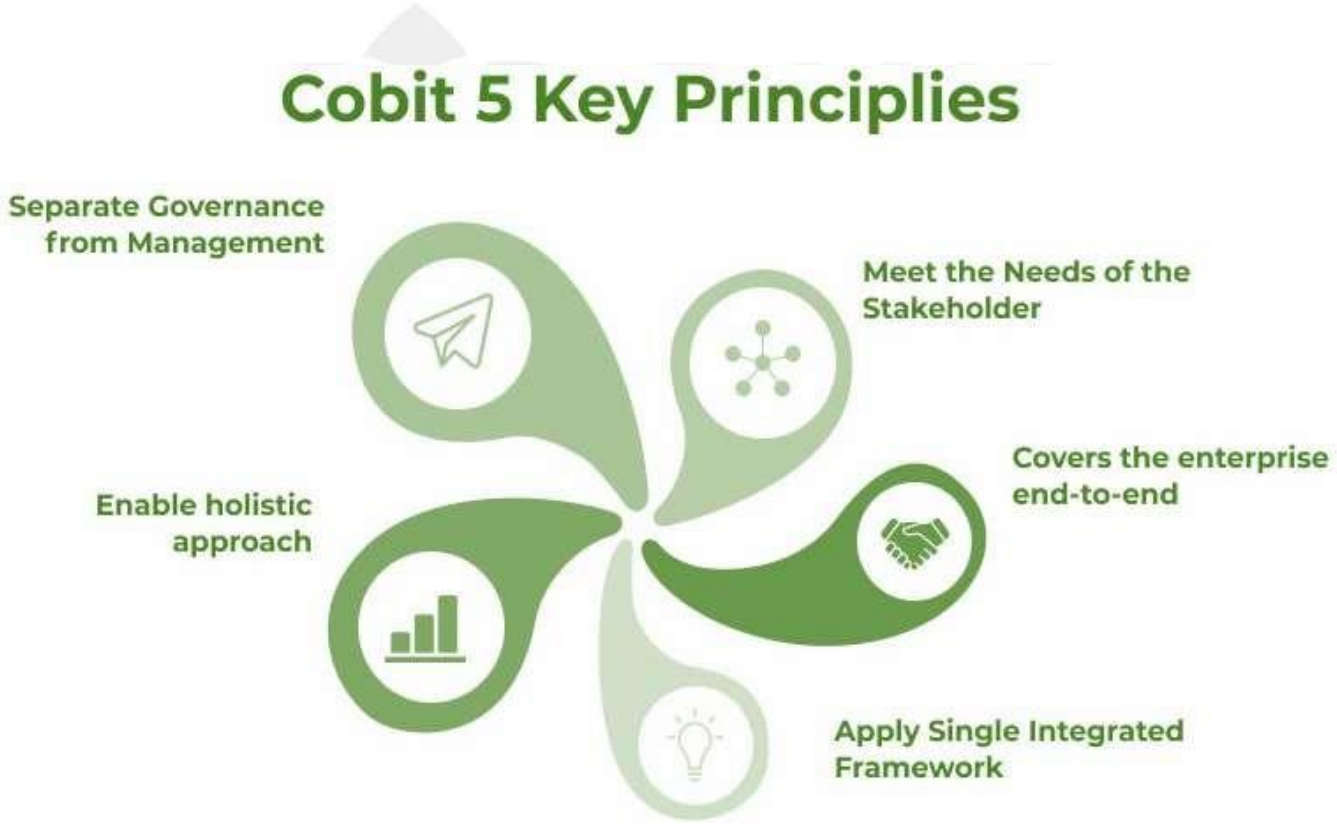
COBIT is used by following group of Peoples

COBIT 2019 is an enterprise framework, three types of people typically engage with the COBIT framework directly:

Management : COBIT helps enterprise managers balance risk versus reward and control investments in an ever-changing IT world.

Auditors : COBIT aids auditors in realizing an acceptable opinion on the rate of assurance on the subject matter being audited and offers advice to management on internal controls.

Users : Enterprise users – typically in-house IT employees – may engage with COBIT principles in order to ensure the security and controls of IT services provided by internal or external parties.



R.M.K GROUP OF INSTITUTIONS

1.2 Systems Methodologies

System methodology is a systematic approach to developing systems efficiently. It involves organizing the best ways to develop systems by describing the work to be done at each stage of the development process. There are multiple methodologies available, and they can differ based on viewpoint.

The System Development Life Cycle (SDLC), is a methodology for the design and implementation of software and hardware to successfully fit a particular set of requirements. It provides a structure that breaks the key aspects of project development into 6 stages, focused on first clearly identifying all requirements, before designing a system with functionalities to match those requirements. The SDLC is intended to help project managers avoid traditional points of failure in the development process, and create impactful technologies that stand the test of time.

LEAN Methodology

Lean methodology is a set of principles and practices that help businesses optimize their processes. The goal of lean methodology is to create value for customers while also creating an efficient workplace.

Lean methodology continues to be popular with businesses due to its focus on **reducing waste and improving efficiency** in the interest of providing more value to customers. Lean methodology was developed by Toyota founder Kiichiro Toyoda after World War II to conserve resources and eliminate waste. It's used in many fields, including healthcare and **software development**.

Lean methodology is based on the following principles:

- Identifying value
- Mapping the value stream
- Creating flow
- Establishing a pull system
- Striving for continuous improvement
- Building quality in
- Creating knowledge
- Deferring commitment
- Respecting people
- Optimizing the whole

Lean methodology in software development helps teams to

- Deliver faster
- Use resources more efficiently
- Produce higher-quality output
- Provide more value to clients and users
- Identify waste and refine processes
- Facilitate efficiency, effectiveness, and continuous improvement

Lean methodology is now considered an integral part of the Agile software development methodology.

R.M.K GROUP OF INSTITUTIONS

There are 7 established lean principles that come with a set of tactics, practices, and processes that build more efficient software products:

1. Eliminating the Waste

To identify and eliminate wastes e.g. unnecessary code, delay in processes, inefficient communication, issues with quality, data duplication, more tasks in the log than completed, etc. regular meetings are held by Project Managers. This allows team members to point out faults and suggest changes in the next turn.

2. Fast Delivery

Previously long-time planning used to be the key to success in business, but with time, it has been found that engineers spend too much time on building complex systems with unwanted features. So they came up with an MVP strategy which resulted in building products quickly that included a little functionality and launching the product to market and seeing the reaction. Such an approach allows them to enhance the product based on customer feedback.

3. Amplify Learning

Learning is improved through ample code reviewing and meetings that are cross-team applicable. It is also ensured that particular knowledge isn't accumulated by one engineer who's writing a particular piece of code so paired programming is used.

4. Builds Quality

LSD is all about preventing waste and keeping an eye on not sacrificing quality. Developers often apply test-driven programming to examine the code before it is written. Quality can also be gained by getting constant feedback from team members and project managers.

R.M.K GROUP OF INSTITUTIONS

5. Respect Teamwork

LSD focuses on empowering team members, rather than controlling them. Setting up a collaborative atmosphere, keeping perfect balance when there are short deadlines and immense workload. This method becomes very important when new members join a well-established team.

6. Delay the Commitment

In traditional project management, it often happens when you make your application and it turns out to be completely unfit for the market. LSD method recognizes this threat and makes room for improvement by postponing irreversible decisions until all experiment is done. This methodology always constructs software as flexible, so new knowledge is available and engineers can make improvements.

7. Optimizing the Whole System

Lean's principle allows managers to break an issue into small constituent parts to optimize the team's workflow, create unity among members, and inspire a sense of shared responsibility which results in enhancing the team's performance.

LSD Process

Here is the overview of the lean software development process:

Identify Value: Understand the customer values and focus on delivering features that meet these needs.

Map the Value Stream: This involves mapping out the entire software development process to identify and eliminate wasteful activities that do not add value.

Create Flow: Ensure a smooth and continuous flow of work by minimizing delays and interruptions.

Establish Pull: Develop features based on customer demand rather than pushing features through the process.

R.M.K GROUP OF INSTITUTIONS

Seek Perfection: Regularly review and refine the development process. Always encourage the team members to identify the areas of improvement and implement changes iteratively.

Build Quality In: Use practices such as test-driven development (TDD) and continuous integration to integrate quality assurance throughout the development process.

Empower Teams: Empower development teams by providing them with the necessary tools, resources, and autonomy to make decisions.

Benefits of LSD

Here are some key benefits of LSD that help organizations to improve their software development processes and outcomes:

Increased Efficiency: LSD reduces delays and inefficiencies by identifying and eliminating non-value-adding activities.

Higher Quality: It integrates quality assurance throughout the development process, thus preventing defects and ensuring quality products.

Faster Delivery: Shorter development cycles allow for quicker release of features and updates, thus meeting customer demands more rapidly.

Adaptability: Delaying decisions until they are necessary and are based on facts, allowing teams to adapt to changes and new information.

Enhanced Collaboration: Engages customers throughout the development process, ensuring that their needs and feedback are continuously addressed.

Limitations of LSD

Here are some key limitations of LSD:

Cultural Resistance: Implementing LSD requires a significant cultural shift and if there is resistance to change from team members and management then it can hinder its adoption and effectiveness.

R.M.K GROUP OF INSTITUTIONS

Learning Curve: There is a steep learning curve associated with understanding and applying lean principles and practices effectively.

Requires Strong Leadership: Successful implementation of LSD requires strong and committed leadership to guide the transition.

Difficulty in Measuring Waste: In LSD, determining waste is subjective and challenging. It requires a deep understanding of processes and value streams.

Resource Intensive: Implementing LSD requires an initial investment in training, tools, and process redesign.

Difference between Lean Development Model and Agile Development Model :

S.No.	AGILE MODEL	LEAN MODEL
1	Agile is more focused on continuous delivery through the self organizing cross-functional teams.	Lean is focused on flow of value to the customer.
2	It is based on the worth and scope of the product.	It is based on how quickly things are developed and how well they turn out i.e. product quality.
3	Aim- To meet the needs of the end user or stakeholder by providing valuable product.	Aim- Elimination of any process that does not have any significant role to the process of producing the final product.
4	Agile is always adaptive to change.	Lean is about stable workflow and meeting the customer's demands.
5	Agile is designed using different frameworks like scrum, Kanban etc.	Lean is designed using transitions from manufacturing processes.

1.3 Development, Release and Deployment Concepts

In software development, the concepts of development, release, and deployment are related to the process of moving software from development to production and making it available to users:

Development: The process of creating the software.

Release: The strategic decision to make the software available to users, and when and to whom.

Deployment: The technical process of moving the software from development to a production environment.

Version Control

In the world of software development, version control, also known as source control, plays a critical role in ensuring a smooth and efficient coding process. With version control, developers can track, manage, and collaborate on changes to their code, resulting in better code management and increased productivity.

Version control provides developers with the ability to recover from mistakes easily. By keeping a record of all changes made to the code, developers can revert back to previous versions if needed, preventing any potential issues or setbacks. This feature proves especially beneficial when working on complex projects with multiple contributors.

One of the key advantages of version control is the ability to review past changes. Developers can easily navigate through the code history, check who made specific changes, and understand the reasoning behind them. This not only fosters collaboration among team members but also helps in troubleshooting and improving the codebase.

R.M.K GROUP OF INSTITUTIONS

Test Driven Development

Test Driven Development (TDD) is a software development practice where developers write automated tests before writing the actual code that needs to be tested. Developers create unit test cases before developing the actual code. It is an iterative approach combining Programming, Unit Test Creation, and Refactoring.

The following sequence of steps is generally followed:

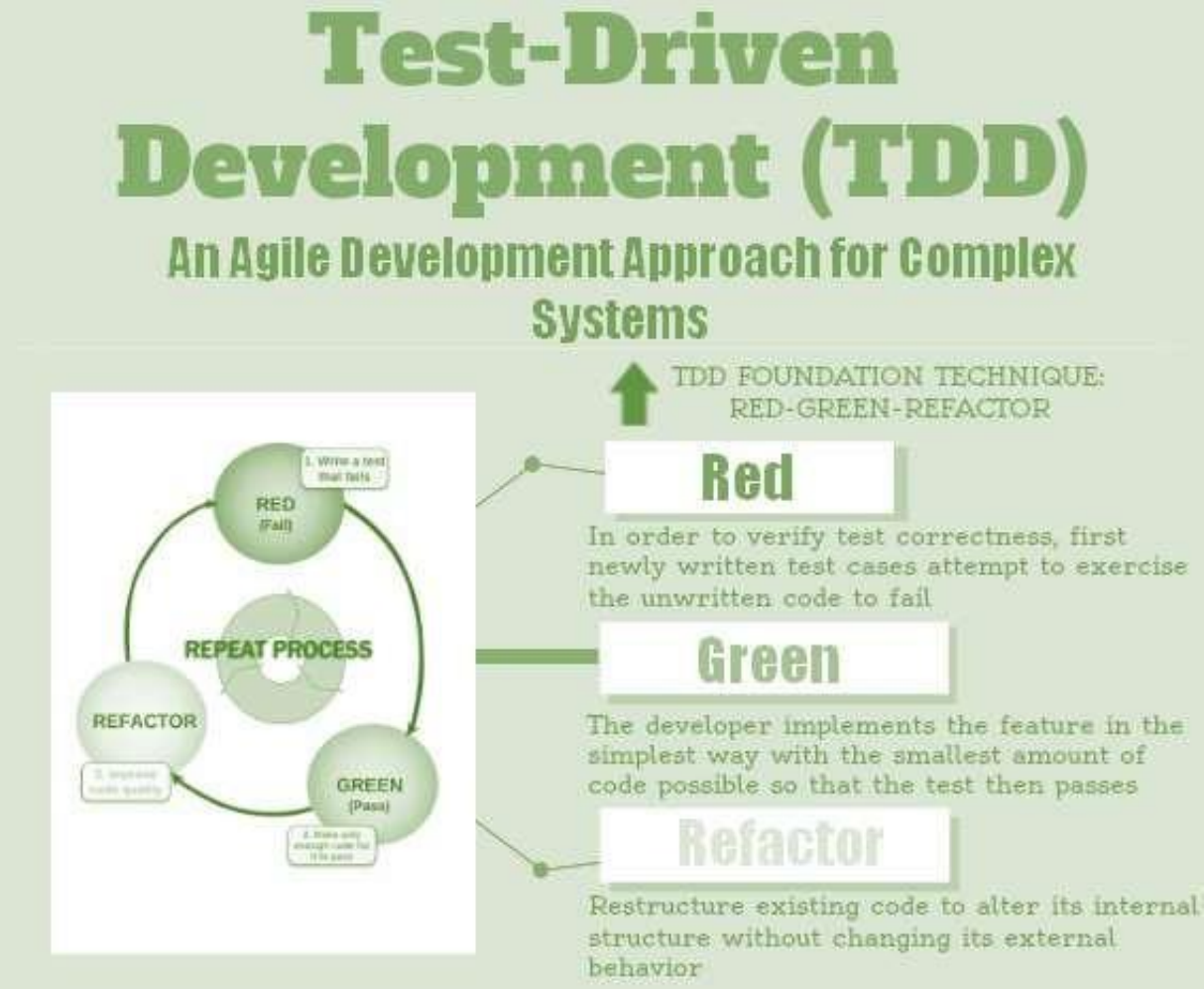
Run all the test cases and make sure that the new test case fails.

Red – Create a test case and make it fail, Run the test cases

Green – Make the test case pass by any means.

Refactor – Change the code to remove duplicate/redundancy. Refactor code – This is done to remove duplication of code.

Repeat the above-mentioned steps again and again



What are the 5 steps of TDD?

1. Add a test, which will certainly FAIL. (Red) ...
2. Run all the tests. See if any test fails. ...
3. Write only enough code to pass all the tests. (Green) ...
4. Run all the tests. If any test fails, go back to step 3. ...
5. Refactor the code. (Refactor) ...
6. If a new test is added, repeat from step 1.

Advantages of Test Driven Development (TDD)

1. Unit test provides constant feedback about the functions.
2. Quality of design increases which further helps in proper maintenance.
3. Test driven development act as a safety net against the bugs.
4. TDD ensures that your application actually meets requirements defined for it.
5. TDD have very short development lifecycle.

R.M.K GROUP OF INSTITUTIONS

Application Deployment

Application deployment is the process of making a software application available for use by end-users. It is also known as software deployment.

Application deployment is a critical phase in the software development lifecycle. It involves several steps, including: Configuring the application, Packaging it into a deployable format, Transferring it to the target environment, and Setting it up for operation.

Some organizations use a deployment model called continuous delivery, which allows software releases to be deployed to production automatically.

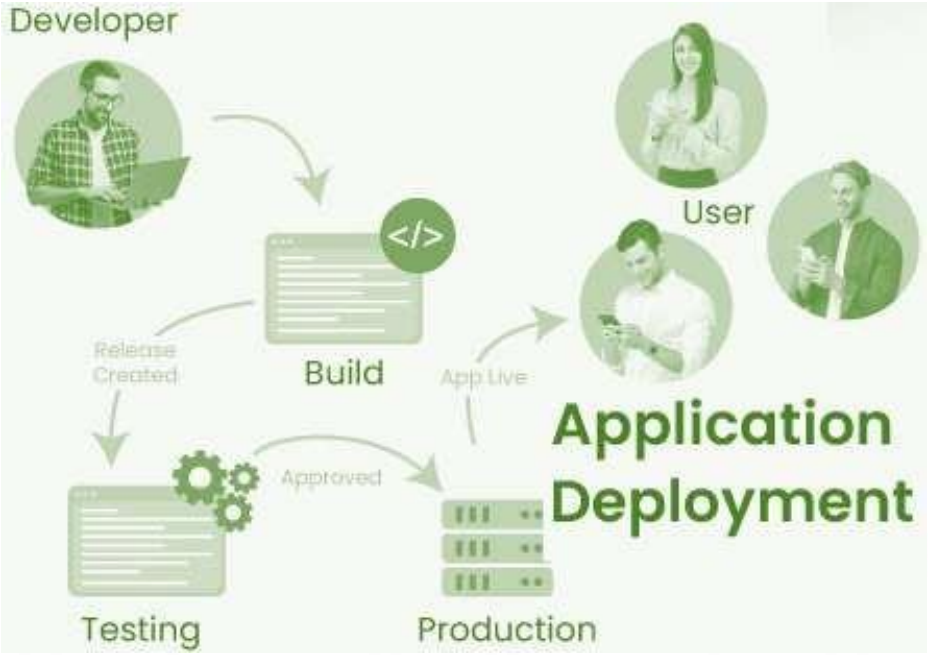
Some other things to consider when deploying an application include:

Monitoring : Continuously monitoring the health and performance of the deployed application.

Rolling back : Being able to roll back a deployment if something goes wrong.

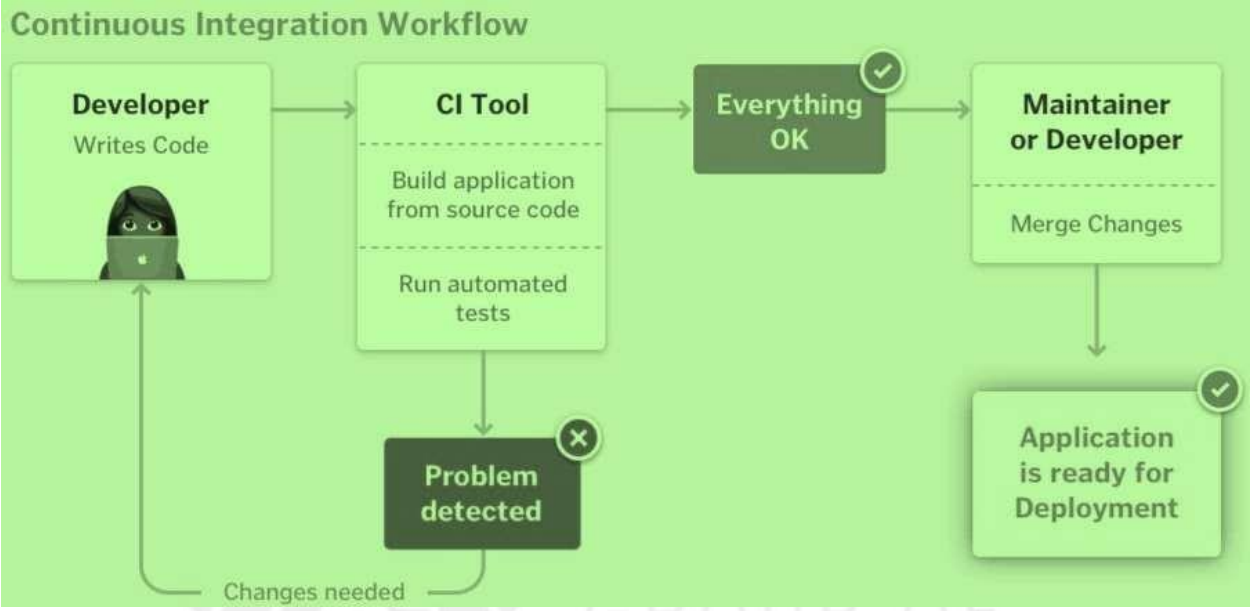
A/B testing : Running multiple versions of an application simultaneously to evaluate which performs best.

Monetization : Deciding how to monetize the app, such as through in-app purchases, ad revenue, or upfront payments



Continuous Integration

Continuous integration (CI) is the practice of automating the integration of code changes from multiple contributors into a single software project. It's a primary DevOps best practice, allowing developers to frequently merge code changes into a central repository where builds and tests then run. Automated tools are used to assert the new code's correctness before integration.



Benefits

1. Improved team productivity and efficiency
2. Accelerated speed to market
3. Better product/market fit
4. Higher quality, more stable products
5. Increased customer satisfaction
6. Happier, more productive developers

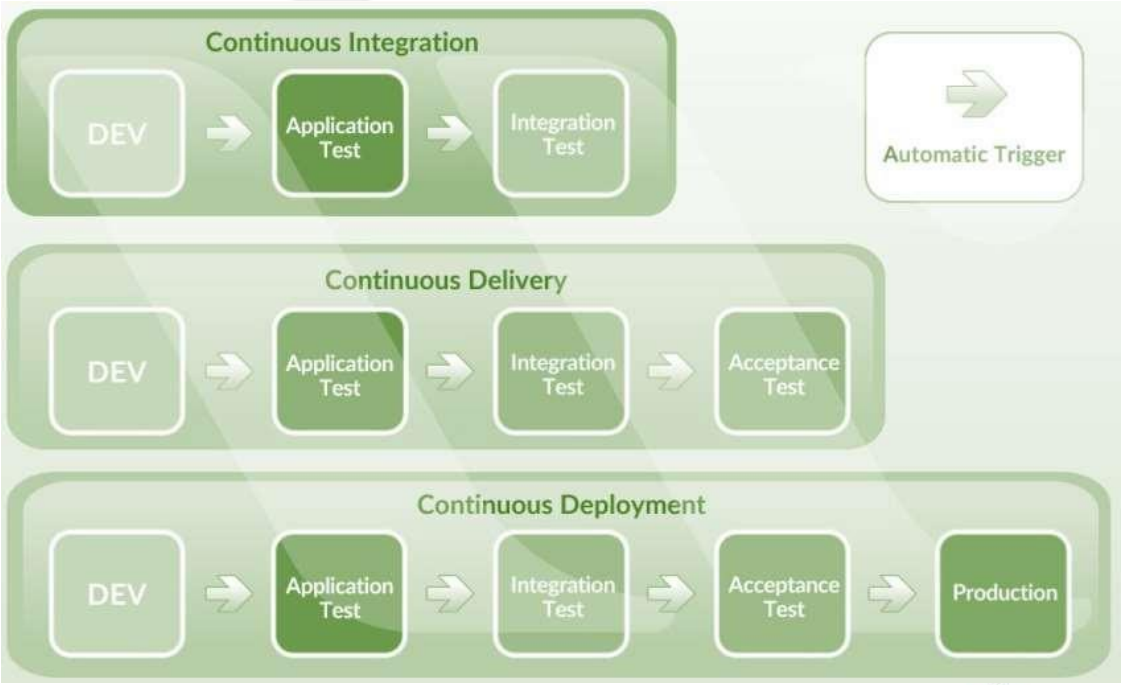
Continuous Delivery

Continuous delivery is a software development practice where code changes are automatically prepared for a release to production. A pillar of modern application development, continuous delivery expands upon continuous integration by deploying all code changes to a testing environment and/or a production environment after the build stage. When properly implemented, developers will always have a deployment-ready build artifact that has passed through a standardized test process.

Continuous Deployment

Continuous deployment is a software development strategy that automatically releases code changes to a production environment. It's part of the continuous integration and continuous delivery (CI/CD) process, which also includes continuous integration (CI).

Continuous deployment aims to minimize the lead time between when a developer writes new code and when it's used by live users. It uses a series of predefined tests to drive the automation. Once new updates pass the tests, the system pushes them directly to the users



Minimum Viable Product

In software development, MVP stands for Minimum Viable Product. It's a technique that involves releasing a product with just enough features to attract early customers and gather feedback for future development.

Characteristics of an MVP:

Just enough features: An MVP has the minimum features required to satisfy early customers.

Quick release: The goal of an MVP is to quickly release a product to market.

Feedback: An MVP enables feedback for future development.

Testing: Testing is a key part of an MVP. It helps to test if an idea works in a market environment while minimizing expenditure.

Risk reduction: An MVP reduces the risk of innovation by allowing for gradual expansion models.

R.M.K GROUP OF INSTITUTIONS

1.4 Infrastructure Concepts

IT infrastructure is the foundation for deploying, operating, and managing a company's technology resources and applications. It's the collection of hardware, software, networks, facilities, and related services that deliver IT operations.

IT infrastructure components include servers, storage systems, networking devices, operating systems, databases, and other software applications. IT infrastructure underpins nearly every aspect of modern business operations.

Components of IT infrastructure

IT infrastructure includes the following elements:

Hardware consists of servers, computers, routers, switches, and storage systems.

Software includes operating systems, applications, and problem management tools.

Networking comprises routers, switches, and cables.

Data centers are facilities housing critical IT equipment and support infrastructure, including power, cooling, and security systems.

Cloud services encompass remote computing services offering scalable IT resources over the internet, such as storage and processing power.

Configuration Management

Configuration management (CM) is a process that ensures a product's performance, functionality, and physical attributes are consistent with its design, requirements, and operational information throughout its life. CM is used in the technology world to manage IT systems and components. It helps to ensure that systems perform as expected and are maintained in a desired state. CM is often used in conjunction with IT service management (ITSM).

R.M.K GROUP OF INSTITUTIONS

Configuration management is a process for maintaining a desired state of IT systems and components. It helps ensure that a system consistently performs as expected throughout its lifecycle.

System administrators can use configuration management tools to set up an IT system, such as a server or workstation, then build and maintain other servers and workstations with the same settings. They can also use configuration assessments and drift analyses to continuously identify systems that have strayed from the desired state and need to be updated, reconfigured, or patched.

Infrastructure automation: configure and manage server infrastructure to enforce consistency and eliminate configuration drift.

Cloud automation: configure and manage cloud resources including operating systems, security groups, load balancers, and virtual private clouds.

Network automation: configure and manage network devices such routers and switches.

Security automation: configure and manage security devices such as firewalls and intrusion detection systems

Edge automation: configure and manage remote infrastructure systems including network, security, IoT devices, and server equipment.

Cloud Computing

Cloud computing is the delivery of computing resources over the internet, or "the cloud", on-demand. It allows users to access computing resources like storage, databases, networking, and software without having to own and maintain physical servers and data centers. Instead, users pay for what they use, which can help lower operating costs and scale more efficiently.

R.M.K GROUP OF INSTITUTIONS

Artifact Management

Artifacts management tools, also known as artifact repositories, are used to store, organize and distribute artifacts (that is, binary files plus their metadata) in a single centralized location. This reduces the amount of time spent downloading dependencies from a public place.

In software development, an artifact refers to any output or deliverable produced during the development process. It can be any file or set of files used, generated, or modified while creating a software application. Artifacts can take various forms, including:

Source code

Compiled code

Libraries and dependencies

Configuration Files

Documentation

Executable Files

Database schemas and scripts

Artifact management in DevOps refers to handling and organizing the various software artifacts produced during the software development life cycle. These artifacts can include compiled code, libraries, documentation, and any other files or assets that are part of a software project.

Containers

Containers allow devs to package everything required to run an app—the code, runtimes, and system tools—into one convenient location. This permits developers to create the applications and deploy them on servers with the operating system itself, which makes containers very lightweight and portable.

R.M.K GROUP OF INSTITUTIONS

Software containers are a key part of modern DevOps practices, and are used to:

Automate tasks

Container orchestration platforms like Kubernetes automate container deployment, scaling, and management.

Improve collaboration

Containers and Kubernetes help development and operations teams collaborate more seamlessly.

Reduce overhead

Containers share an operating system, so they require less overhead and start up faster than virtual machines (VMs).

Provide isolation

Containers provide application-level isolation, with each container having its own file system, processes, and network stack.

Secure containers

Containers share the host operating system's kernel, so it's important to protect them from vulnerabilities and attacks.

R.M.K GROUP OF INSTITUTIONS

1.5 What is DevOps?

DevOps is a tactical approach, not a specific technology. The term is a combination of the words "development" and "operations".

DevOps is a set of practices, tools, and cultural philosophies that combine software development and IT operations into a single process.

DevOps originated around 2007 when the software development and IT operations communities recognized the need to improve on the traditional model. In this model, developers and operations teams worked separately, with developers writing code and operations deploying and supporting it.

DevOps teams often work together across the entire application lifecycle, from development to operations. They use a variety of tools and technologies to operate and evolve applications quickly and reliably. DevOps teams aim for high availability, strong security, zero downtime, and system reliability

R.M.K GROUP OF INSTITUTIONS

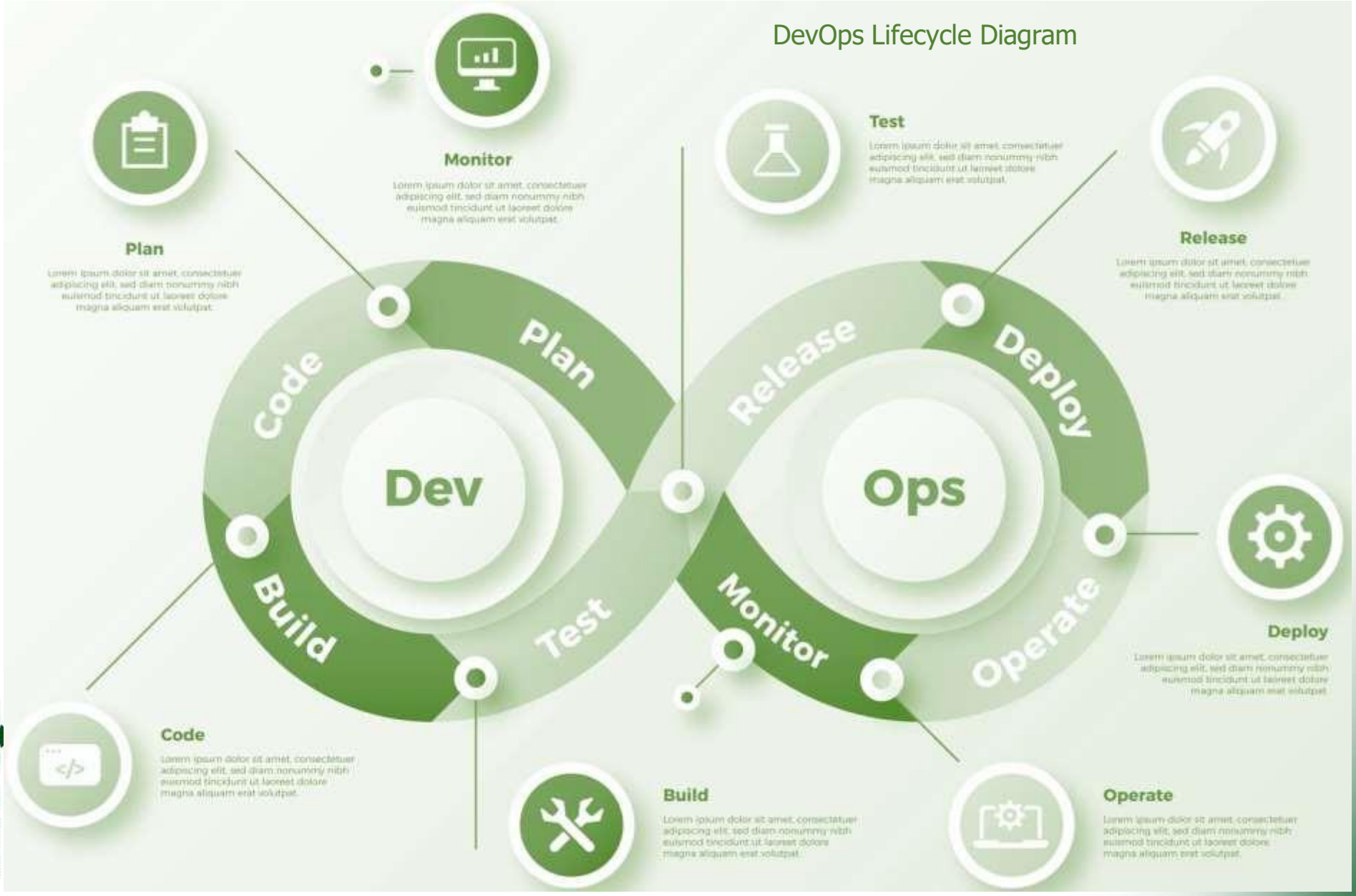
DevOps Life Cycle

The DevOps lifecycle is a continuous software development process that employs DevOps best practices to plan, build, integrate, deploy, monitor, operate, and offer continuous feedback throughout the software's lifecycle. It is often represented by a continuous loop diagram as shown in the diagram.

The DevOps lifecycle is a series of collaborative and continuous practices that help bridge the gap between software development and IT operations

R.M.K GROUP OF INSTITUTIONS

DevOps Lifecycle Diagram



R.M.K GROUP OF INSTITUTIONS

Phases of the DevOps lifecycle

Plan: The planning phase is exactly what it sounds like: planning the project's lifecycle. In contrast to conventional methods to the development lifecycle, this model assumes that each stage will be repeated as necessary. In this manner, the DevOps workflow is planned with the likelihood of future iterations and likely prior versions in mind.

This implies that we will likely have information from past iterations that will better inform the next iteration, and that the present iteration will likewise inform the next iteration.

Code: The developers will write the code and prepare it for the next phase during the coding stage. Developers will write code in accordance with the specifications outlined in the planning phase and will ensure that the code is created with the project's operations in mind.

Build: Code will be introduced to the project during the construction phase, and if necessary, the project will be rebuilt to accommodate the new code. This can be accomplished in a variety of ways, although GitHub or a comparable version control site is frequently used.

The developer will request the addition of the code, which will then be reviewed as necessary. The request will be approved if the code is ready to be uploaded, and the code will be added to the project. Even when adding new features and addressing bugs, this method is effective.

Test: Throughout the testing phase, teams will do any necessary testing to ensure the project performs as planned. Teams will also test for edge and corner case issues at this stage. An "edge case" is a bug or issue that only manifests during an extreme operating event, whereas a "corner case" occurs when many circumstances are met.

R.M.K GROUP OF INSTITUTIONS

Release: The release phase occurs when the code has been verified as ready for deployment and a last check for production readiness has been performed. The project will subsequently enter the deployment phase if it satisfies all requirements and has been thoroughly inspected for bugs and other problems.

Deploy: In the deploy phase, the project is prepared for the production environment and is operating as planned in that environment. This would be the responsibility of the operations team; in DevOps, it is a shared responsibility. This shared duty pushes team members to collaborate to guarantee a successful deployment.

Operate: In the operating phase, teams test the project in a production environment, and end users utilise the product. This crucial stage is by no means the final step. Rather, it informs future development cycles and manages the configuration of the production environment and the implementation of any runtime requirements.

Monitor: During the monitoring phase, product usage, as well as any feedback, issues, or possibilities for improvement, are recognized and documented. This information is then conveyed to the subsequent iteration to aid in the development process. This phase is essential for planning the next iteration and streamlines the pipeline's development process.

R.M.K GROUP OF INSTITUTIONS

Importance and Benefits of DevOps

Faster delivery

DevOps can help organizations reduce the time it takes to get new features and bug fixes to customers.

Improved quality

DevOps can help organizations produce higher quality software with fewer bugs.

Better collaboration

DevOps can help organizations break down silos between development and operations teams, leading to better communication and collaboration.

Increased reliability

DevOps can help organizations ensure the quality of application updates and infrastructure changes.

Greater scalability

DevOps can help organizations manage complex or changing systems efficiently and with reduced risk.

Improved security

DevOps can help organizations incorporate security practices earlier in the software development lifecycle.

Faster time to market

DevOps can help organizations reduce the overall time from conception to deployment.

Increased revenue

DevOps can help organizations increase revenue by quickly fixing defects and adding new features that customers request.

More productive teams

DevOps can help organizations create more productive and efficient teams.

Happier customers

DevOps can help organizations create happier customers.

R.M.K GROUP OF INSTITUTIONS

1.6 DevOps Principles and Practices

Customer-centric action

DevOps teams use short feedback loops with customers and end users to develop products and services centered around user needs. DevOps practices enable rapid collection and response to user feedback through use of real-time live monitoring and rapid deployment. Teams get immediate visibility into how live users interact with a software system and use that insight to develop further improvements.

Create with the end in mind

This principle involves understanding the needs of customers and creating products or services that solve real problems. Teams should not create software based on assumptions about how consumers will use the software. Rather, DevOps teams should have a holistic understanding of the product, from creation to implementation.

Automation

An essential practice of DevOps is to automate as much of the software development lifecycle as possible. This gives developers more time to write code and develop new features. Automation is a key element of a CI/CD pipeline and helps to reduce human errors and increase team productivity. With automated processes, teams achieve continuous improvement with short iteration times, which allows them to quickly respond to customer feedback.

Continuous Improvement

Continuous improvement was established as a staple of agile practices, as well as lean manufacturing and Improvement Kata. It's the practice of focusing on experimentation, minimizing waste, and optimizing for speed, cost, and ease of delivery. Continuous improvement is also tied

R.M.K GROUP OF INSTITUTIONS

to continuous delivery, allowing DevOps teams to continuously push updates that improve the efficiency of software systems. The constant pipeline of new releases means teams consistently push code changes that eliminate waste, improve development efficiency, and bring more customer value.

Cross Functional Autonomous Team

A cross-functional autonomous team is a group of people with different skills and expertise who work together to achieve a common goal while having the autonomy to make decisions. Cross-functional teams are flexible and don't necessarily have a designated leader. Team members are equally responsible for contributing to the team's goal. Cross-functional teams can lead to innovation, efficiency, and better communication.

DevOps Practices

Continuous Integration/Delivery/Deployment

A change at any stage of the software delivery pipeline, from development to production systems, should go through a CI/CD system. This ensures that if anything fails, there is a fast feedback loop leading to rapid recovery.

Flexibility

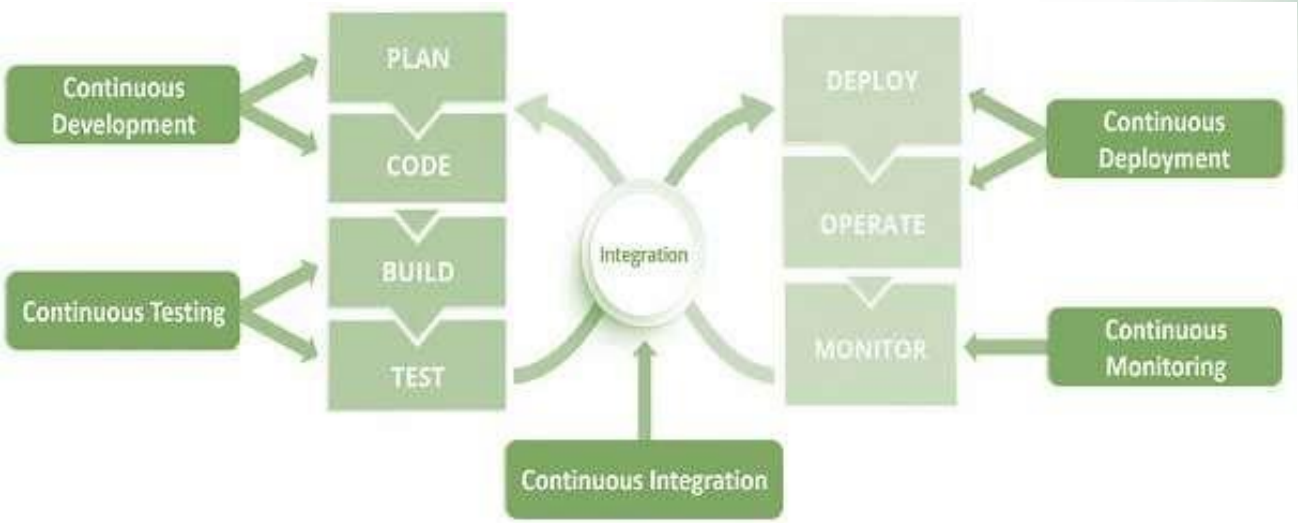
DevOps flexibility allows to scale up or down operations quickly, which is critical. With the right approach, we can scale our operations to meet increased demand, increase security to handle system failures, and provide customers with a consistent experience, regardless of their channel of communication.

Agile Decision Making

Agile DevOps is a strategic approach that combines Agile software development methodologies with DevOps practices to speed up the feedback loop between developers and operations teams. Agile decision-making processes emphasize collaboration and teamwork to solve problems and progress projects.

R.M.K GROUP OF INSTITUTIONS

7C's of DevOps Life Cycle



1. Continuous Development: This step is crucial in defining the vision for the entire software development process. It focuses mostly on project planning and coding. At this phase, stakeholders and project needs are gathered and discussed. In addition, the product backlog is maintained based on customer feedback and is divided down into smaller releases and milestones to facilitate continuous software development.

Once the team reaches consensus on the business requirements, the development team begins coding to meet those objectives. It is an ongoing procedure in which developers are obliged to code whenever there are modifications to the project requirements or performance difficulties.

2. Continuous Integration: Continuous integration is the most important stage of the DevOps lifecycle. At this phase, updated code or new functionality and features are developed and incorporated into the existing code. In addition, defects are spotted and recognized in the code at each level of unit testing during this phase, and the source code is updated accordingly. This stage transforms integration into a continuous process in which code is tested before each commit. In addition, the necessary tests are planned during this period.

R.M.K GROUP OF INSTITUTIONS

3. Continuous Testing: Some teams conduct the continuous testing phase prior to integration, whereas others conduct it after integration. Using Docker containers, quality analysts regularly test the software for defects and issues during this phase. In the event of a bug or error, the code is returned to the integration phase for correction. Moreover, automation testing minimises the time and effort required to get reliable findings. During this stage, teams use technologies like as Selenium. In addition, continuous testing improves the test assessment report and reduces the cost of delivering and maintaining test environments.

4. Continuous Deployment: This is the most important and active step of the DevOps lifecycle, during which the finished code is released to production servers. Continuous deployment involves configuration management to ensure the proper and smooth deployment of code on servers. Throughout the production phase, development teams deliver code to servers and schedule upgrades for servers, maintaining consistent configurations.

In addition to facilitating deployment, containerization tools ensure consistency throughout the development, testing, production, and staging environments. This methodology enabled the constant release of new features in production.

5. Continuous Feedback: Constant feedback was implemented to assess and enhance the application's source code. During this phase, client behaviour is routinely examined for each release in an effort to enhance future releases and deployments. Companies can collect feedback using either a structured or unstructured strategy. Under the structural method, input is gathered using questionnaires and surveys. In contrast, feedback is received in an unstructured manner via social media platforms. This phase is critical for making continuous delivery possible in order to release a better version of the program.

R.M.K GROUP OF INSTITUTIONS

6. Continuous Monitoring – During this phase, the functioning and features of the application are regularly monitored to detect system faults such as low memory or a non-reachable server. This procedure enables the IT staff to swiftly detect app performance issues and their underlying causes. Whenever IT teams discover a serious issue, the application goes through the complete DevOps cycle again to determine a solution. During this phase, however, security vulnerabilities can be recognized and corrected automatically.

7. Continuous Operations – The final phase of the DevOps lifecycle is essential for minimizing scheduled maintenance and other planned downtime. Typically, developers are forced to take the server offline in order to perform updates, which increases the downtime and could cost the organization a large amount of money. Eventually, continuous operation automates the app's startup and subsequent upgrades. It eliminates downtime using container management platforms such as Kubernetes and Docker.

R.M.K GROUP OF INSTITUTIONS

	Tools		
DevOps Phases	Continuous Planning & Development	• GitLab • GIT • TFS • SVN • Mercurial • Jira • BitBucket • Trello	• Maven • Gradle • Confluence • Subversion • Scrum • Lean • Kanban
	Continuous Integration	• Jenkin • Bamboo • GitLab CI	• TeamCity • Travis and CircleCI • Buddy
	Continuous Testing	• JUnit • Selenium • JMeter • Cucumber • TestSigma	• Microfocus UFT • TestNG • Tricentis Tosca • Jasmine
	Continuous Deployment	• Ansible • Chef • Docker • IBM Urban Code • Kubernetes	• Puppet • Go • Vagrant • Spinnaker • ArgoCD
	Continuous Monitoring	• Nagois • Grafa na • Kibana • Prometheus • Logstash • AppDynamics	• ELK Stack • New Relic • Splunk • Sensu • PagerDuty
	Customer Feedback	• Webalizer • W3Perl • ServiceNow • Slack	• Flowdock • Open Web Analytics • Pendo • Qentelli's TED
	Continuous Operations	• Kubernetes	• Docker Swarm

1.7 DevOps and Continuous Testing

Continuous testing is a key part of DevOps, a software development methodology that uses agile practices to streamline development:

What is continuous testing?

Continuous testing (CT) is a practice that automatically runs tests throughout the software development lifecycle (SDLC). CT helps ensure quality and functionality at every stage of development, from development to deployment.

Continuous testing CT working in DevOps

Continuous testing CT tools are preinstalled with testing scripts that run automatically whenever new code is integrated into the application. CT tests can include unit, integration, system, and performance tests.

Benefits of CT in DevOps

CT helps teams catch and fix defects early, reducing the risk of costly issues in production. It also provides quick feedback on business risks associated with each release candidate.

How CT fits into the DevOps pipeline?

CT is a linchpin in the software delivery pipeline, working hand in hand with continuous integration (CI) to automatically validate any new code.

Benefits of Continuous Testing in DevOps

- Early discovery of critical bugs
- Seamless collaboration among developers, QA and Operations team
- Helps to assess the quality of software developed at each stage
- Can be seamlessly incorporated into DevOps
- Helps drive faster test results which leads to improved code quality
- Repeated testing ensures minimal failure rate for new releases
- Faster time to market with a viable product and continuous feedback mechanism

R.M.K GROUP OF INSTITUTIONS

Challenges in Implementing Continuous Testing

Integration with DevOps: Many teams struggle to implement continuous testing within Agile and DevOps frameworks due to a lack of the right tools or training.

Cultural Shift: Adopting continuous testing often requires a significant change in mindset, particularly for teams used to traditional testing methods.

Updating Testing Strategies: Relying on old testing approaches or poorly managed test data can reduce the effectiveness of continuous testing.

Frequent Code Integration: Developers who don't integrate their code frequently, ideally multiple times a day, risk creating issues like duplicated efforts or incompatible code, leading to defects.

Test Environment Management: It's essential to ensure that the test environments are aligned with the latest code in the repository to enable smooth testing.

Production Environment Alignment: For thorough testing, the production environment must closely match the test environment to ensure all software aspects are adequately validated.

Implementing Continuous Testing Strategy in DevOps

1.8 How to choose right DevOps tools?

https://anarsolutions.com/how-to-choose-the-right-devops-tools/

Checklist to keep in mind while choosing the right DevOps tools.

Step 1: Collaboration

As your team is sharing ideas and planning at the start of each sprint, take into account feedback from the previous sprint's retrospective so the team and your services are continuously improving. Tools can help you centralize learnings and ideas into a plan. To kick off the brainstorming process, you can identify your target personas, and map customer research data, feature requests, and bug reports to each of them. We like to use physical or digital sticky notes during this process to make it easier to group together common themes and construct user stories for the backlog. By mapping user stories back to specific personas, we can place more of an emphasis on the customer value delivered. **Top tools we can use :** Active Collab, Pivotal Tracker, VersionOne, Jira, Trello, StoriesOnBoard

Step 2: Use tools to capture every request

No changes should be implemented outside the DevOps process. All types of requests for any changes in the software or any new additions to the software should be captured by the DevOps. It provides automation to the system to accept requests for change that may inflow from both the sides either the business or the DevOps team. For instance, making changes to the software to facilitate a request to improve the access to the database.

Step 3: Usage of Agile Kanban project management

A primary advantage that Kanban has is that it encourages teams to focus on improving flow in the system. As teams adopt Kanban, they become good at continuously delivering work they have completed. Thus,

Kanban facilitates doing incremental product releases with small chunks of new functionality or defect fixes. This feature of Kanban makes it very suited to DevOps' continuous delivery and deployment requirements.

The other big advantage of Kanban is how it enables you to visualize your entire value-stream and ensure stable flow. Thus, it helps you combine the workflows of different functions and activities right from Dev to Integration/ Build, Test, Deployment and beyond that to application monitoring. Initially, it will help your Dev and Ops people to work in a collaborative manner. Over a period of time, you can evolve into a single team and single workflow that includes all of Dev and Ops activities. Kanban provides you visibility to this entire process – and transformation to a DevOps culture.

Step 4: Usage of tools to Log Metrics

One should always opt for the tools that help in understanding the productivity of the DevOps processes both in automated and in manual processes. From this one can determine if it is working in the favor or not. First decide which metrics are more relevant to the DevOps processes, like speed to effective action vs. errors occurred. Secondly, use an automated process to rectify issues without the help of a human controller. For instance, dealing with problems related to scaling of software automatically on digital cloud podium.

Step 5: Implementation

Automated testing is just a minor part of automated testing. Test automation is an ability to implement code and data and the solution thus obtained to ensure its high quality. A continuous series of test is must do with DevOps.

Step 6: Acceptance tests

It is necessary to perform Acceptance tests as they help in a deployment of each part thereby acceptance of the infrastructure. The testing process should also define the degree of acceptance tests that are to be a part of

R.M.K GROUP OF INSTITUTIONS

apps, data and test suite. A good amount of time must be spent in testing and retesting of DevOps and define acceptance tests to ensure that the tests are in sync with the criteria selected. As applications evolve over time, new instructions are fed and again the testing should be done.

Automated Testing

Automated testing pays off over time by speeding up your development and testing cycles in the long run. And in a DevOps environment, it's important for another reason: awareness.

To prepare for and support what Development builds, it's important for Operations to have visibility into what is being tested, and how thoroughly. Unlike manual tests, automated tests are executed faithfully and with the same rigor every time. They also yield reports and trend graphs that help identify risky areas.

Risk is a fact of life in software, but you can't mitigate what you can't anticipate. Do your operations team a favor and let them peek under the hood with you. Look for tools that support wallboards and let everyone involved in the project comment on specific build or deployment results. Extra points for tools that make it easy to get Operations involved in blitz testing and exploratory testing.

Tools we can use: Bamboo, Bitbucket, Capture for Jira

Step 7: Continuous Feedback

Continuous feedback focuses on providing ongoing feedback and coaching by openly discussing an employee's strengths and weaknesses on a regular basis. Feedback is of utmost importance to spot gaps and inefficiencies' in the app. Feedback loops are of great help for automated conversation among the tests. The right tool should be able to spot any issue using manual or automated mechanisms. A collaborative approach towards solving the problem should be adopted for achieving impeccable results.

1.9 Challenges with DevOps implementation

1. Change in Culture

The workplace culture undergoes the major amount of transformation while implementing DevOps. It is also one of the most difficult areas of transformation as it is a long term process which also requires a lot of patience and endurance. To make the process a bit easier, enterprises should try and maintain a positive as well as a transparent atmosphere in the workplace.

2. Switching from Legacy Infrastructure to Microservices

In order to reduce stability issues, organisations now use infrastructure as code along with microservices for quicker development along with sharp innovations. Moreover, organisations need to update their hardware and software systems according to the latest trends on a regular basis, so that new systems can co-exist with the existing systems.

3. Issues with the standards and metrics

Dev and Ops departments have different goals and working systems, hence they have different toolsets as well. It might become very tedious to sit together and integrate the tools. Under such circumstances, it is advisable that the teams agree upon a commonly decided metric system.

4. Tool Turbulence

Switching to DevOps practices might make people dependent on the various tools that are available to solve even the smallest of their problems. Due to this, organisations might become addicted to those tools which provide with short-term benefits over the ones which provide with long-term benefits. Some of the tools are open-sourced or SaaS-based and can be easily adopted without any authorization. To make things easier, you can provide teams with a set of library tools from which they can opt for their preferred tools. This will also help the leaders stay up-to-date with the activities of the employees.

R.M.K GROUP OF INSTITUTIONS

5. Resistance to Change

You might come across people in your company who might not be supportive of the legacy systems. They are the ones who have become comfortable with their way of working and are not willing to leave their comfort zones. Hence, it is very important that you don't bend down to such elements but instead bear with the discomfort of change.

6. Challenges during the process

Adopting DevOps can prove to be challenging for workers who blindly follow guidelines and stay stuck to the rules, or for companies which follow specific guidelines for software development, as DevOps doesn't have any fixed framework stating procedures that employees can follow to reach their desired goals.

The teams can decide on their own course of action without any structural approach, giving them opportunities and more scope for innovation.

7. Test Automation

Test Automation holds equal importance as CI/CD deployments. It has been commonly observed that companies tend to neglect test automation and focus more on CI/CD deployments. For DevOps to be a success, continuous testing acts as a key.

8. Cost and Budget

It is very important to keep in mind that open source does not necessarily mean that it is free of cost. Moreover, factor in integration and operational complexity to your overall costs.

10. Assignment Questions

S.No.	Topic	K - Level	COs
1	How is DevOps different from agile methodology?	K3	CO1
2	How will you approach a project that needs to implement DevOps?	K3	CO1

R.M.K GROUP OF INSTITUTIONS

11. Part A
Question & Answer

R.M.K
GROUP OF
INSTITUTIONS

R.M.K
GROUP OF
INSTITUTIONS

Part A

1. What is DevOps? (CO1, K1)

In general, DevOps is the gray area between development (Dev) and operations (Ops) teams in a product development process. DevOps is a culture in which communication, integration, and collaboration in the product development cycle are emphasized.

2. Which are some of the most popular DevOps tools? (CO1, K1)

The most popular DevOps tools include:

- Selenium
- Puppet
- Chef
- Git
- Jenkins
- Ansible
- Docker

3. What are the different phases in DevOps? (CO1, K1)

The various phases of the DevOps lifecycle are as follows:

Plan: Initially, there should be a plan for the type of application that needs to be developed. Getting a rough picture of the development process is always a good idea.

Code: The application is coded as per the end-user requirements.

Build: Build the application by integrating various codes formed in the previous steps.

Test: This is the most crucial step of the application development. Test the application and rebuild, if necessary.

Integrate: Multiple codes from different programmers are integrated into one.

Deploy: Code is deployed into a cloud environment for further usage. It is ensured that any new changes do not affect the functioning of a high traffic website.

Operate: Operations are performed on the code if required.

Monitor: Application performance is monitored. Changes are made to meet the end-user requirements.

R.M.K GROUP OF INSTITUTIONS

Part A

4. What is continuous integration? (CO1, K1)

Continuous integration (CI) is the process of automatically integrating code changes from multiple developers into a shared repository. Automated tests are utilized to verify and affirm the additional codes generate no conflict with the existing codebase. Ideally, code changes should be merged multiple times a day, at every commit, with the help of CI tools.

5. What is continuous delivery? (CO1, K1)

Continuous delivery (CD), together with CI makes a complete flow for deliverable code packages. In this phase, automated building tools are applied to compile artifacts (e.g., source code, test scripts, configuration files, and environments) and have them ready to be delivered to the end user. With that in mind, in a CD environment, new releases are just one click away from being published with fully functional features and minimal human intervention.

6. What is continuous testing? (CO1, K1)

Continuous testing is the practice of applying automated tests early, gradually, and adequately in the software delivery pipeline. In a typical CI/CD workflow, builds are released in small batches. Therefore, it is impractical to manually perform test cases for each delivery. Automated continuous testing eliminates the manual steps and turns them into automated routines, which reduces human effort. That's why automated continuous testing is essential for the DevOps culture.

7. What are the and its benefits? (CO1, K1)

Benefits of continuous testing:

- Ensures the quality and speed of builds
- Enables faster software delivery and a continuous feedback mechanism
- Detects errors as soon as they occur in the system
- Reduces business risks and evaluates potential problems before they become real problems

8. What is version control and its uses? (CO1, K1)

Version control (or source control) is a repository in which all changes in the source code are stored and managed at all times. Version control provides an operating history of code development that allows developers to rewind if they made any mistakes, compare versions, and finalize the best version of the code.

Part A

9. What are some benefits of continuous delivery? (CO1, K1)

Manually releasing changes in code gives teams complete control of the product. In some cases, a new product release aligns with a bigger picture in the organization, for example, a promotion strategy with clear business purposes.

By automating repetitive and mundane tasks, IT professionals have more time to focus on improving the product without worrying about the integration progress.

10. What are some benefits of continuous deployment? (CO1, K1)

With continuous deployment, developers can focus solely on the product because their final task in the pipeline is to review pull requests and merge them to the master branch. This method enables frictionless deployment and shortens the deployment duration by releasing new features and fixes right after they have passed the automated tests.

Customers are the ones to evaluate the quality of each release. Bug fixes for new releases are easier to handle because each release is delivered in small batches.

11. Mention some of the Technical benefits of DevOps. (CO1, K2)

- Continuous software delivery
- Less complex problems to manage
- Early detection and faster correction of defects

12. Mention some of the Business benefits of DevOps. (CO1, K2)

- Faster delivery of features
- Stable operating environments
- Improved communication and collaboration between the teams

13. What is Docker, and why is it used?

Docker is an open-source platform that enables developers to create, deploy, and run applications within lightweight, portable containers. These containers package an application along with all of its dependencies, libraries, and configuration files.

Part A

14. How does DevOps differ from traditional software development methodologies? (CO1, K1)

DevOps differs from traditional software development methodologies in its collaboration, automation, and continuous delivery. Instead of having separate teams for development and operations, DevOps promotes cross-functional teams that work together to streamline the entire software development process.

15. Name three important DevOps KPIs. (CO1, K1)

Three of the most common DevOps KPIs are:

- Meantime to failure recovery
- Deployment frequency
- Percentage of failed deployments

16. What is the importance of continuous feedback in DevOps? (CO1, K1)

Continuous feedback is essential in DevOps as it helps teams identify and address issues early, improving the quality and speed of software delivery. By collecting feedback throughout the software development process, teams can continuously monitor the performance of their applications and infrastructure and make improvements as needed.

17. List out the 7Cs of the DevOps Lifecycle (CO1, K1)

i. Continuous development.
ii. Continuous integration.
iii. Continuous testing.
iv. Continuous deployment/Continuous delivery.
v. Continuous monitoring.
vi. Continuous operations.
vii. Continuous feedback.

R.M.K GROUP OF INSTITUTIONS

Part A

18. What is COBIT? (CO1, K1)

COBIT is a collection of best practices and principles, along with tools and models for making sure IT resources and processes align with and support business goals. COBIT (Control Objectives for Information and Related Technologies) is a framework that helps organizations manage and govern their IT resources. It provides a set of guidelines, principles, and best practices to help organizations.

19. What is ITIL Methodology? (CO1, K1)

ITIL stands for Information Technology Infrastructure Library. ITIL was developed in the 1980s by the United Kingdom's Central Computer and Telecommunications Organization (CCTA), a government agency responsible for providing IT assistance to other ministries.

20. What is CI/CD? (CO1, K1)

CI And CD is the practice of automating the integration of code changes from multiple developers into a single codebase. It is a software development practice where the developers commit their work frequently to the central code repository (Github or Stash).

21. What is Infrastructure as Code (IaC)? (CO1, K1)

Infrastructure as Code (IaC) is a method of managing and provisioning IT infrastructure using code, rather than manual configuration. It allows teams to automate the setup and management of their infrastructure, making it more efficient and consistent. This is particularly useful in the DevOps environment, where teams are constantly updating and deploying software.

R.M.K GROUP OF INSTITUTIONS

Part A

22. What Is Jenkins? (CO1, K1)

Jenkins is a tool that is used for automation, and it is an open-source server that allows all the developers to build, test and deploy software. It works or runs on java as it is written in java. By using Jenkins we can make a continuous integration of projects(jobs) or end-to-endpoint automation.

23. Is DevOps a tool? (CO1, K2)

DevOps is not a tool or a technology, it is a set of practices aimed at breaking down silos between the Development and Operations team.

24. What are the benefits of automation testing?

Automation testing has several advantages, including quicker and more effective testing, expanded coverage, and higher test accuracy. It can save time and money in the long run because automated testing can be repeated without human intervention.

25. What is the role of cloud computing in DevOps? (CO1, K2)

Cloud computing provides a centralized, scalable communication platform for DevOps teams at each stage of the software development lifecycle. It allows team members to collaborate more quickly and closely.

R.M.K GROUP OF INSTITUTIONS

12. PART – B Questions

1. How will you approach a project that needs to implement DevOps? (CO1, K2)
2. What is the difference between continuous delivery and continuous deployment? (CO1, K2)
3. Explain all the phases in DevOps in detail. (CO1, K2)
4. Explain the term "Infrastructure as Code" (IaC) as it relates to configuration management.
5. Why Has DevOps Gained Prominence over the Last Few Years?
6. What are the fundamental differences between DevOps & Agile?
7. What are the benefits of Automation Testing?
8. How to choose the right DevOps tools? - Challenges with DevOps implementation.
9. Explain about Software Development Methodologies.
10. What are Operations Methodologies
11. What are Systems Methodologies

Note : All the above questions requires Diagrams to Illustrate the Concepts, Wherever Necessary.

R.M.K GROUP OF INSTITUTIONS

13. Supportive online courses

Online Courses

1. https://www.coursera.org/learn/intro-to-devops
2. https://www.udemy.com/course/decodingdevops/?couponCode=IND21PM

External Links for Additional Resources

1. https://www.geeksforgeeks.org/devops-tutorial/
2. https://www.simplilearn.com/tutorials/devops-tutorial/devops-interview-questions#what_is_devops
3. https://aws.amazon.com/devops/what-is-devops/
4. https://azure.microsoft.com/en-us/products/devops
5. https://roadmap.sh/devops

R.M.K GROUP OF INSTITUTIONS

14. Real Time Applications

1. Real Time CI/CD Pipeline for Java Application to Deploy on Apache Server
2. Building Scalable, Secure, and High-Performance Web Applications with AWS 3-Tier Architecture
3. CICD PROJECT: Production Level Blog APP Deployment using EKS, Nexus, SonarQube, Trivy with Monitoring Tools
4. CI/CD Project: Deploy a 3-tier Microservice Voting App using ArgoCD and Azure DevOps Pipeline
5. DevSecOps: OpenAI Chatbot UI Deployment in EKS with Jenkins and Terraform
6. DevSecOps: Deploy Reddit App to Amazon Elastic Kubernetes Service (EKS) using ArgoCD and monitor its performance
7. DevOps Project to Automate Infrastructure on AWS Using Terraform and GitLab CICD
8. DevSecOps (DevOps) Project: Deploying a Petshop Java-Based Application with CI/CD, Docker, and Kubernetes
9. From Scratch to Production: Deploying EKS Clusters and Applications with CI/CD using Jenkins and Terraform
10. Real Time DevOps Project : Deploy to Kubernetes Using Jenkins : End to End DevOps Project : CI/CD

R.M.K GROUP OF INSTITUTIONS

15. Content Beyond Syllabus

What is a DevOps Engineer?

A DevOps engineer is an expert in developing, deploying, and maintaining software systems using the DevOps methodology and practices.

DevOps engineers collaborate closely with IT operations teams, software developers, and other stakeholders to guarantee the effective delivery of software products. To increase the efficiency and quality of software development, they implement automation, continuous integration, and continuous delivery/deployment (CI/CD) practices. They also locate and resolve issues that arise throughout the development process.

DevOps engineers often have extensive backgrounds in IT operations, systems administration, software development, scripting, automation, and cloud computing skills. They are adept at using various tools and technologies, including source code management systems, build and deployment tools, virtualization and container technologies, and monitoring and logging tools.

DevOps engineers must be proficient in technical and interpersonal skills, teamwork, and problem-solving techniques. They also need to be able to interact and collaborate successfully with coworkers from all backgrounds and disciplines since they work closely with several teams within the business.

15. Content Beyond Syllabus

What are the Requirements to Become a DevOps Engineer?

Different criteria for becoming a DevOps engineer may exist depending on the business and the individual function. However, some specific fundamental skills and certifications are frequently needed or recommended. They consist of the following:

Excellent technical background: DevOps engineers should be well-versed in IT operations, systems administration, and software development. A degree in computer science, information technology, or a similar discipline may be required, as well as relevant experience and certifications.

Experience with DevOps tools and methodologies: DevOps engineers should have experience with various DevOps technologies and processes, including version control systems, build and deployment automation, containerization, cloud computing, and monitoring and logging tools.

Scripting and automation skills: DevOps engineers should have strong scripting skills and be proficient in using tools such as Bash, Python, or PowerShell to automate tasks and processes.

Cloud computing experience: DevOps engineers should have experience working with cloud platforms such as Amazon Web Services (AWS), Microsoft Azure, or Google Cloud Platform (GCP).

Soft skills: DevOps engineers should be effective communicators, able to collaborate with teams across departments, and possess strong problem-solving and analytical skills.

Certification: Some organizations may require DevOps engineers to hold relevant certifications such as Certified DevOps Engineer (CDE), Certified Kubernetes Administrator (CKA), or AWS Certified DevOps Engineer - Professional.

R.M.K GROUP OF INSTITUTIONS

16. Assessment Schedule

Tentative schedule for the Assessment During 2024-2025 Even Semester

S. No.	Name of the Assessment	Start Date	End Date	Portion
1	Unit Test 1	399		Unit 1
2	IAT 1			Unit 1 & 2
3	Unit Test 2			Unit 3
4	IAT 2			Unit 3 & 4
5	Revision 1			Unit 5, 1 & 2
6	Revision 2			Unit 3 & 4
7	Model			All 5 Units

INSTITUTIONS

17. Text Books & References

TEXTBOOKS:

1. Deepak Gaikwad, Viral Thakkar, "DevOps Tools: from Practitioner's Point of View", Wiley, 2019.
2. Jennifer Davis, Ryn Daniels, "Effective DevOps", O'Reilly Media, 2016.

REFERENCES:

1. Gene Kim, Jez Humble, Patrick Debois, "The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations", IT Revolution Press, 2016.
2. Jez Humble, Gene Kim, "Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation", Addison-Wesley, 2010.
3. Yevgeniy Brikman, "Terraform: Up & Running: Writing Infrastructure as Code", O'Reilly Media, 2019.
4. Joseph Muli, "Beginning DevOps with Docker", Packt Publishing, 2018.

18. Mini Project Suggestions

1. Deploy Java Application on AWS 3-Tier Architecture. (CO1, K3)
2. Deploy Scalable VPC Architecture on AWS Cloud. (CO1, K3)
3. Deploy your code on a Docker Container using Jenkins on AWS. (CO1, K3)
4. DevSecOps : Netflix Clone CI-CD with Monitoring Email. (CO1, K3)
5. Zomato Clone: Secure Deployment with DevSecOps CI/CD. (CO1, K3)

R.M.K GROUP OF INSTITUTIONS

Thank you

Disclaimer:

This document is confidential and intended solely for the educational purpose of RMK Group of Educational Institutions. If you have received this document through email in error, please notify the system manager. This document contains proprietary information and is intended only to the respective group / learning community as intended. If you are not the addressee you should not disseminate, distribute or copy through e-mail. Please notify the sender immediately by e-mail if you have received this document by mistake and delete this document from your system. If you are not the intended recipient you are notified that disclosing, copying, distributing or taking any action in reliance on the contents of this information is strictly prohibited.